AI in Cyber Security

Unit 1 Mini Project — Technical Report

Author	Rudraraj Radhwani
Dataset	Aposemat IoT-23, capture CTU-IoT-Malware-Capture-1-1
Source	Stratosphere Laboratory, CTU University (CC BY 4.0)
Sample	84,063 Zeek conn.log flow records
Scope	Tasks 1-4 (Task 5, ethical reflection, not included)
Repository	https://github.com/Rudraraj24/ai-cybersecurity-unit1-mini-project-rudraraj

1. Problem Overview

Organisations increasingly depend on IoT devices that are rarely patched and often ship with default credentials, which makes them attractive targets for botnet recruitment. Signature-based tools struggle here because the traffic itself is unremarkable: a compromised device does not send malformed packets, it simply opens a very large number of ordinary connections. This project builds an AI-enabled analysis and detection prototype on real labelled network traffic and asks a specific question: **do the reported accuracy figures on this dataset actually measure threat detection?**

The dataset is a four-day packet capture from a single infected IoT device. Each row is a Zeek connection summary — metadata only, no payload — labelled Benign or Malicious by the Stratosphere Laboratory. Published work on this capture routinely reports accuracy above 99.9%. A central finding of this report is that such figures are obtainable without any machine learning at all.

2. Methodology

2.1 Preprocessing

Missing values. The columns *duration*, *orig_bytes* and *resp_bytes* are absent on the same 78.94% of rows — those where the connection was never answered. A missing value here therefore means “no reply”, which is a different event from “answered with zero bytes”. Filling both with zero would merge two opposite outcomes. The pipeline imputes with the median but adds a binary was-missing indicator column, so the distinction survives into the model.

Categorical encoding. One-hot encoding, with levels occurring fewer than 30 times collapsed into a single “infrequent” category. The *history* field has over 100 rare values; without collapsing them the models receive near-empty columns that allow memorisation of individual flows.

Normalisation. StandardScaler on all numeric features. Logistic Regression and the neural networks are scale-sensitive and the byte counts span six orders of magnitude.

Train/test split. Temporal, not random — the first 70% by timestamp is training data. Adjacent flows are near-duplicates of the same scan burst, so a random split places near-identical rows on both sides and measures memorisation rather than generalisation. A temporal split also matches deployment: a real detector must classify traffic it has not seen.

2.2 Two feature configurations

Every model is trained twice. The **full** configuration uses all modellable columns. The **honest** configuration removes three: *id.resp_p* (destination port), *id.orig_p* (source port) and *service*. Section 3.2 explains why those three have to be treated separately from the rest.

Identifier columns (*id.orig_h*, *id.resp_h*) and the raw timestamp are excluded from both. With 78,596 distinct destination IPs, a model given that column memorises hosts; given a timestamp under a temporal split, it learns a date threshold.

3. Results and Observations

3.1 Threat landscape (Task 2)

Protocol	Benign	Malicious	% malicious
icmp	1,471	0	0.00%
tcp	3,710	44,945	92.37%
udp	33,936	1	0.00%

Table 1: Protocol against class label.

Three observations characterise the capture. First, **96.18% of all flows have connection state S0** — an attempt that received no reply whatsoever. That is the signature of horizontal scanning. Second, the protocol split is extreme: TCP is 92.37% malicious while UDP and ICMP are essentially 0%. Third, the device contacts **78,596 distinct destination IPs** across the capture, fanning out to thousands of hosts per five-minute window.

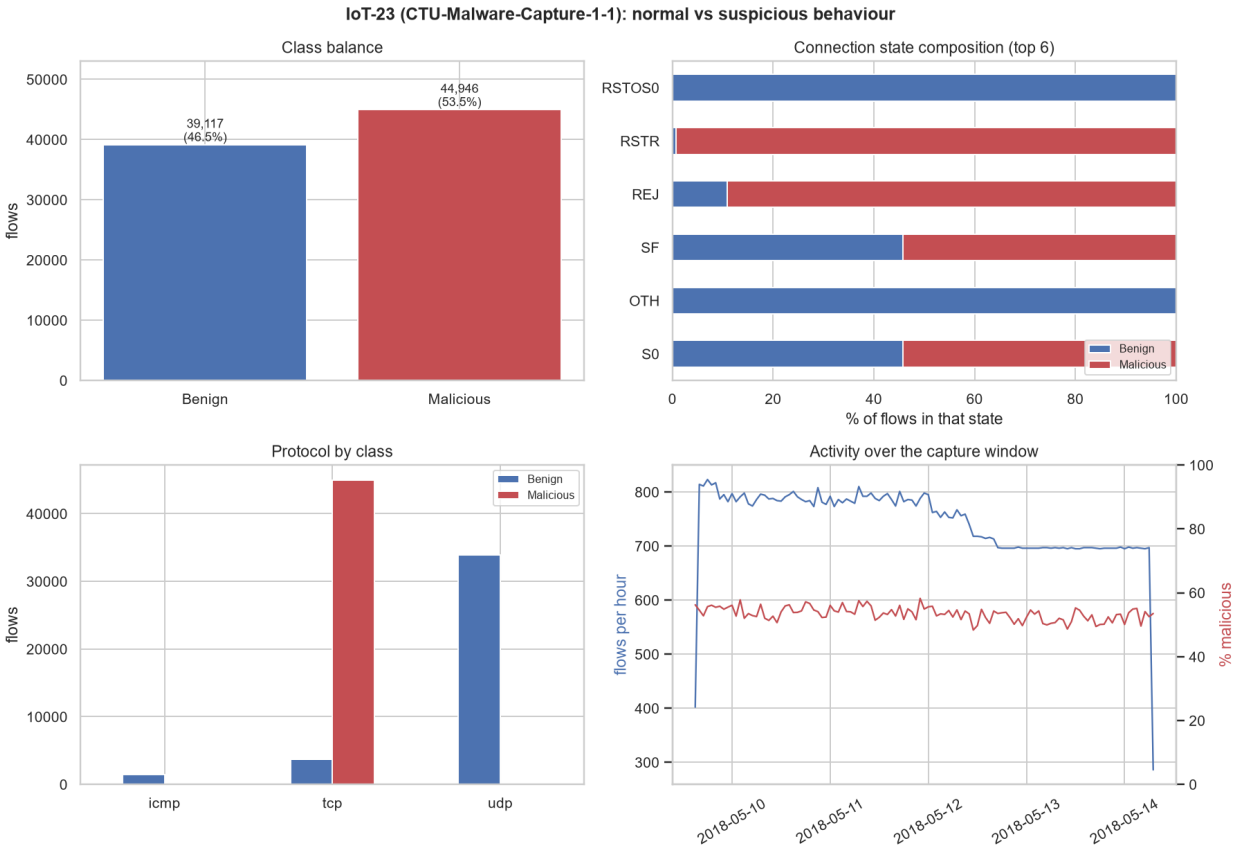


Figure 1: Class balance, connection-state composition, protocol split, and activity across the four-day capture window.

3.2 The labels are derived from the destination port

Four destination ports account for every malicious flow. A hand-written rule using nothing else reproduces the ground truth almost perfectly:

Rule	TP	FP	FN	TN	Accuracy	Precision	Recall
id.resp_p in {23, 2323, 8080, 9527}	44,945	0	1	39,117	99.9988%	100.0000%	99.9978%

Table 2: A four-entry lookup table, scored on the test period.

Zero false positives. The Stratosphere team labelled a flow as scan traffic when enough flows converged on a single port, so the destination port is not a predictor of the label — it *is* the label. Any model given that column recovers the labelling rule rather than learning to detect anything, which is why the three port-derived columns are separated out in the honest configuration. A second consequence is that “Benign” does not mean legitimate: most benign rows are unanswered connection attempts from the same infected device that fell below the flagging threshold.

3.3 Model comparison (Tasks 3 and 4)

Model	Type	Accuracy	Precision	Recall	F1	PR-AUC	FP	Train (s)
LogisticRegression	ML	99.01%	0.9813	1.0000	0.9906	0.9945	250	0.47
DecisionTree	**ML**	**100.00%**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**0**	**0.45**
MLP-small	DL	99.01%	0.9814	1.0000	0.9906	0.9945	249	2.32
MLP-deep	DL	99.62%	0.9927	1.0000	0.9963	0.9980	97	27.46

Table 3: Full feature set. The algorithms separate clearly.

The Decision Tree wins on every axis at once — perfect accuracy, zero false positives, and joint-fastest training. The reason is structural rather than incidental: the true decision boundary in this dataset is a *rule*, and a tree represents rules natively, finding it exactly in a handful of splits. A neural network must approximate the same step function with continuous weights, which is why the 14,593-parameter MLP-deep spends roughly 25-55 times the training time and still lands short at 99.62%. The usual accuracy-versus-interpretability trade-off does not appear here; the interpretable model is simply better.

Model	Type	Accuracy	Precision	F1	PR-AUC	FP
LogisticRegression	ML	95.55%	0.9213	0.9590	0.9244	1,121
DecisionTree	ML	95.55%	0.9213	0.9590	0.9211	1,121
MLP-small	DL	95.55%	0.9213	0.9590	0.9192	1,121
MLP-deep	DL	95.55%	0.9213	0.9590	0.9243	1,121

Table 4: Honest feature set. The algorithms become indistinguishable.

Remove the three port-derived columns and all four models return **identical results to four decimal places**, and every one of them agrees with the single-feature rule *proto* == *tcp* on 100.000% of test predictions. They did not learn to detect threats; they learned that in this capture the TCP traffic is the scanning traffic.

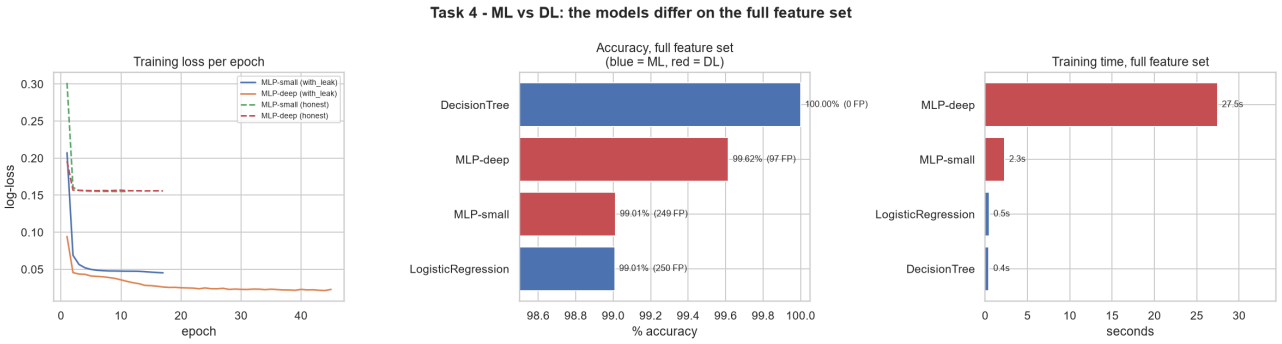


Figure 2: Training loss, accuracy and training time across all four models on the full feature set.

3.4 Verifying that the identical scores are real

Four models returning the same number is the signature of a copy-paste error, so it was tested three ways before being reported.

Experiment	What was changed	Outcome
1. Threshold sweep	Decision threshold swept across the full range	Each model emits only 17-42 distinct probabilities across 25,219 rows. No threshold separates them.
2. Drop the dominant feature	Restricted to TCP, making proto useless	All models return recall 1.0000 and FN 0 — they predict “malicious” for every TCP flow and collapse to the majority class.
3. Remove class imbalance	Training set balanced by undersampling	Models finally differ, but all sit within ±0.01 of the no-skill PR-AUC baseline (0.9214) while accuracy falls as low as 7.86%.

Table 5: Three independent checks on the identical scores.

The scores are not a coding error and not a coincidence. Once the destination port is removed, Zeek's flow metadata in this capture carries **approximately one usable bit — the protocol** — and every algorithm finds it immediately. Experiment 3 is the clearest: balancing produces variation between models without producing any skill, so the differences reflect how each algorithm fails rather than what any of them learned.

4. Conclusion

Flow-level supervised classification on this capture is not a solved detection problem but a mislabelled one. Given the leaking columns the four algorithms separate cleanly and a Decision Tree reaches 100% by reproducing the labelling rule exactly. Take those columns away and all four converge on the same single-feature rule and the same 1,121 false positives. **The algorithms are only distinguishable while they have access to the answer key.**

The reported 99.9% figures for this dataset are therefore reproducible but not meaningful: they measure how the labels were generated. The appropriate response is not a larger model — experiment 3 shows additional capacity buys nothing — but better features. Per-host windowed behaviour (fan-out, connection rate, share unanswered, destination-IP entropy) is strongly present in this traffic, cannot be reproduced by a port lookup, and is the natural next step.

Limitations

One device, one malware family, one network, under five days. The analysis uses a 1-in-12 systematic sample of the full 1,008,748-flow capture, which retains only one of its eight command-and-control flows, so no multi-class or C&C-detection claim is supported. Results should not be assumed to generalise to other networks or attack types.

Reproducibility

All dependencies are pinned in `requirements.txt` and `random_state=42` is fixed throughout. Training times are wall-clock and vary between runs; every accuracy, precision, recall, F1, PR-AUC and false-positive figure in this report is deterministic and reproduces exactly. Every number here was read programmatically from the result tables in `outputs/tables/` rather than transcribed by hand.